

# 1 SilentWindow

## 1.1 How the project was built — step by step

**Repository:** gokulrajmisox/ews

**Document version:** 1.0

**Implementation baseline:** commit 11e5425

**Status:** Research prototype and retrospective replay system

**Safety boundary:** SilentWindow is not a medical device, does not provide diagnosis or treatment advice, and has not been prospectively validated. The system uses retrospective endpoint/proxy labels rather than a verified timestamp for physiological collapse.

## 1.2 1. The problem the project was built to solve

The challenge is to anticipate patient deterioration from ICU telemetry that is:

- irregularly sampled;
- incomplete because sensors disconnect or observations are absent;
- noisy because of motion, artifacts, and implausible jumps; and
- operationally difficult because excessive alarms create alarm fatigue.

The intended outcome is not “alert on every abnormal number.” It is a chronological early-warning workflow that asks whether a signal is trustworthy, whether a patient is deviating from their own baseline, and whether the evidence has persisted long enough to justify changing state.

SilentWindow therefore targets a three-way trade-off:

| Objective                | Design response                                                          |
|--------------------------|--------------------------------------------------------------------------|
| Handle imperfect data    | Trust-aware credibility scoring, missingness flags, and staleness clocks |
| Preserve real chronology | Past-only feature extraction and chronological replay                    |
| Reduce alarm burden      | Sequential evidence accumulation, refractory periods, and alert budgets  |

## 1.3 2. Product concept

The product is a ward-monitoring research console with three layers:

```
Irregular telemetry
  |
  v
Trust Layer ---> Is this observation credible?
  |
  v
Temporal Features ---> How different is this patient from their baseline?
  |
```

```

      v
XGBoost + calibration ---> What is the estimated risk?
      |
      v
Evidence Accumulator ---> Has risk persisted long enough?
      |
      v
STABLE ---> WATCH ---> ALERT

```

The most important product behavior is the **STABLE** → **WATCH** → **ALERT** progression. A single suspicious spike should not automatically become a high-confidence alert.

### 1.4 3. Step 1 — Inspect and configure the data

The pipeline is configured in `configs/config.yaml`.

Important defaults are:

- raw records under `data/raw/`;
- a 70% / 15% / 15% patient-level train/validation/test split;
- a 6-hour personal baseline window;
- 1-hour replay resolution;
- a maximum 48-hour telemetry horizon;
- configurable trust ranges and jump limits;
- a sigmoid calibration layer; and
- configurable watch/alert thresholds, refractory period, and alert budget.

The raw PhysioNet 2012 files are intentionally not committed. A reproducible training checkout needs:

```

data/raw/Outcomes-train.txt
data/raw/set-a/<RecordID>.txt

```

The application's checked-in model and replay artifacts allow the dashboard and demo API to run without placing restricted raw data in the repository.

### 1.5 4. Step 2 — Split patients before creating model features

The split is done by patient identifier rather than by individual rows. This matters because a single patient contributes many time points. A row-level random split could place the same patient in both training and testing and make the model appear stronger than it really is.

The core split logic lives in `ml/preprocessing.py` and is tested in `tests/test_silentwindow.py`.

The test suite explicitly checks:

```

train n validation = 0
train n test       = 0
validation n test  = 0

```

When the raw dataset is absent, that raw-data-dependent test is skipped with an explicit message rather than failing mysteriously.

## 1.6 5. Step 3 — Build the Trust Layer

The Trust Layer is implemented in `ml/trust_layer.py`.

For each observation, it preserves the original value and returns an assessment containing:

- parameter name;
- timestamp;
- original value;
- validity flag;
- credibility score;
- reason for downweighting; and
- optional cross-parameter context.

The processor checks:

1. configured physiological plausibility ranges;
2. sudden rate-of-change jumps;
3. systolic/diastolic blood-pressure discordance;
4. missingness and time since the last measurement; and
5. credibility penalties from the configuration.

A suspicious reading is not silently deleted. It remains available for audit and is downweighted in downstream evidence accumulation.

Example reasoning:

```
HR = 220
valid = false
credibility = low
reason = physiologically implausible
```

That distinction is important: the system separates **what was observed** from **how much the pipeline trusts it**.

## 1.7 6. Step 4 — Engineer patient-specific temporal features

`ml/features.py` creates features using only observations available at the evaluation time.

The feature extractor builds:

- personal baseline means;
- deviation from baseline;
- rolling history and slopes over 1, 3, and 6 hours;
- missingness indicators;
- time-since-last-measurement values;
- recent variability;
- shock-index-style signals; and
- compound heart-rate, blood-pressure, and respiratory deterioration signals.

The baseline is patient-specific rather than population-only. A value that is unusual for one patient may be normal for another.

### 1.7.1 Causality guarantee

At evaluation time  $T$ , the extractor only uses records where:

```
time_hours <= T
```

The test `test_no_future_leakage_assertion` injects catastrophic future observations after time `T` and verifies that the feature vector at `T` remains identical.

This is one of the project's most important scientific safeguards.

## 1.8 7. Step 5 — Train and calibrate the model

The training path is implemented in `ml/train.py`.

The saved production-style artifact uses:

- an XGBoost tabular classifier;
- 127 saved feature columns;
- patient-level splits;
- a validation calibration stage; and
- a persisted model and calibrator under `models/`.

The runtime loads:

```
models/xgboost_model.joblib
models/calibrator.joblib
models/feature_names.json
```

The model produces a raw probability. The calibration layer converts that value into an empirically calibrated risk estimate before it is passed to the sequential policy.

The project also contains an offline grouped retraining benchmark in `reports/model_improvement_report.md`. That benchmark reported ROC-AUC 0.819 and PR-AUC 0.371, but it is **not** the deployed artifact because the original raw files and the exact production feature contract are not available for a safe drop-in replacement.

## 1.9 8. Step 6 — Accumulate evidence instead of firing on one score

`ml/evidence_accumulator.py` implements the third layer.

The accumulator is CUSUM-inspired. It combines:

- calibrated risk;
- credibility weight;
- persistence of elevated evidence;
- decay when evidence normalizes;
- a refractory period after an alert; and
- a maximum alert budget per patient.

The configured states are:

| State  | Meaning                                          |
|--------|--------------------------------------------------|
| STABLE | No persistent high-confidence evidence           |
| WATCH  | Elevated evidence merits review on the dashboard |

| State | Meaning                                                          |
|-------|------------------------------------------------------------------|
| ALERT | Evidence crossed the configured high-confidence policy threshold |

Tests cover transient-spike rejection, sustained-risk accumulation, refractory suppression, and alert-budget behavior.

## 1.10 9. Step 7 — Evaluate the saved retrospective replay

The saved evaluation is based on a held-out cohort of:

- 152 patients;
- 22 positive endpoint outcomes;
- 130 negative endpoint outcomes;
- 6,943 chronological records; and
- up to 48 hours per patient.

The independently reconstructed saved-model metrics were:

| Metric                     | Result |
|----------------------------|--------|
| ROC-AUC                    | 0.641  |
| PR-AUC                     | 0.244  |
| Accuracy at current policy | 76.3%  |
| Precision                  | 15.0%  |
| Sensitivity                | 13.6%  |
| F1                         | 0.143  |

The saved SilentWindow policy produced 3 true-positive alerts and 51 false-alert events in the saved test run. A threshold baseline produced more positive detections but also many more false-alert events.

These results show a real trade-off; they do not prove clinical superiority.

### 1.10.1 Alert-policy sweep

The new `ml/policy_sweep.py` module replays the saved records at multiple alert thresholds and reports:

- total alerts;
- false-alert events;
- alerted patients;
- positive patients detected;
- sensitivity;
- precision; and
- alerts per patient-day.

The API route is:

```
GET /api/policy-sweep?thresholds=0.55,0.65,0.75,0.85,0.95
```

Threshold selection must happen on validation data before a final locked test report. The endpoint is an analysis tool, not an automatic threshold selector.

## 1.11 10. Step 8 — Add robustness testing

The noise lab is implemented in `ml/noise_test.py` and exposed through:

```
GET /api/noise-lab
POST /api/noise-lab/run
```

The saved stress test injects missingness, spikes, and timestamp jitter. It is explicitly labeled as a **synthetic corruption experiment**, not evidence from naturally arriving hospital telemetry.

The system records the effect of corruption on alert behavior rather than pretending that noisy-data performance is automatically clinically validated.

## 1.12 11. Step 9 — Build the dashboard

The dashboard is served from:

```
frontend/index.html
frontend/app.js
```

It exposes:

- overview metrics;
- ward monitor state and risk;
- patient timeline replay;
- synchronized vital, risk, and evidence charts;
- performance and ablation views;
- noise lab controls; and
- optional legacy Gemini and Telegram integrations.

The replay UI filters points to the selected time before rendering. It is a retrospective visualization, not a live bedside monitor.

The demo video is versioned at:

```
demos/SilentWindow_Yuva_Megathon_Demo.mp4
```

## 1.13 12. Step 10 — Add the incremental inference API

The current runtime includes a live-style simulation API implemented in `backend/streaming.py`.

A client sends one chronological observation batch:

```
curl -X POST http://127.0.0.1:8000/api/v1/patients/138123/observations \
-H 'Content-Type: application/json' \
-d '{
  "timestamp_hours": 4,
  "observations": {"HR": 82, "SysABP": 118, "RespRate": 19},
  "static_info": {"Age": 65, "ICUType": 2}
}'
```

The response includes:

- raw risk;

- calibrated risk;
- evidence score;
- state;
- alert/suppression status;
- credibility and downweighted readings;
- feature count and config path; and
- the research disclaimer.

Supporting routes are:

```
GET    /api/v1/patients/{patient_id}/state
DELETE /api/v1/patients/{patient_id}/state
```

The service rejects:

- empty observation batches;
- non-finite values;
- negative timestamps;
- timestamps beyond the configured horizon; and
- out-of-order batches.

State is process-local and protected by a lock. It is suitable for a local demonstration and shadow-mode prototype, not a durable multi-worker deployment.

## 1.14 13. Step 11 — Harden the runtime

The supported application entry point is now:

```
backend.main:app
```

The root `main.py` remains only as a compatibility wrapper.

The runtime includes:

- GET `/health` for liveness;
- GET `/version` for service, config, and artifact metadata;
- restrictive localhost-default CORS;
- deterministic startup that does not train at server boot;
- bounded legacy file upload size;
- Telegram request timeout; and
- optional AI imports loaded only when the legacy AI route is called.

A clean install is defined in `requirements.txt`. Developer commands are available through Makefile:

```
make install
make test
make check
make run
make reproduce
```

## 1.15 14. Step 12 — Add CI and container deployment

The repository includes:

```
.github/workflows/ci.yml
Dockerfile
.dockerignore
```

CI runs on Python 3.10, 3.11, and 3.12 and performs:

1. dependency installation;
2. Python compilation;
3. unit tests; and
4. API contract tests.

The demo container can be launched with:

```
docker build -t silentwindow .
docker run --rm -p 8000:8000 silentwindow
```

The raw challenge data is excluded from the image. The container is for serving the checked-in artifacts and dashboard; retraining still requires the data separately.

## 1.16 15. Licensing and dataset terms

The SilentWindow source code and project documentation are available under your choice of three alternative licenses:

- MIT License — LICENSE;
- Apache License 2.0 — LICENSE-APACHE; or
- GNU General Public License v3.0 — LICENSE-GPL-3.0.

These are alternative options, not cumulative obligations. The complete, unmodified license texts are included in the repository.

The selected project license covers the project code and documentation only. Third-party dependencies retain their own licenses. None of these project licenses relicenses the PhysioNet / Computing in Cardiology Challenge 2012 dataset. The dataset is not included in this repository; obtain it from its authoritative source and follow its separate access, citation, and usage terms.

## 1.17 16. What was verified

At the implementation baseline used for this guide:

- 13 tests passed;
- 1 raw-data split test was intentionally skipped because data/raw is not in the checkout;
- Python compilation passed;
- health/version endpoint smoke checks passed;
- incremental inference scored a batch successfully;
- out-of-order input was rejected;
- policy-sweep smoke checks passed;
- README local-link checks passed; and
- the branch was synchronized with origin/main.

## 1.18 17. What is not yet solved

The following are still open because they require missing raw data, timestamped labels, prospective telemetry, or production governance:

1. Verified timestamped deterioration targets.
2. Multi-horizon clinical event prediction.
3. Prospective shadow-mode evaluation.
4. Durable multi-worker patient state.
5. Authentication, rate limiting, and a production audit trail.
6. Subgroup calibration and fairness analysis.
7. Clinical review and deployment approval.
8. A safe production integration with hospital telemetry systems.

The system should therefore be presented as a **transparent research prototype that makes the alarm-burden trade-off explicit**, not as a clinically validated predictor.

## 1.19 18. Reproduce the project

### 1.19.1 Demo/runtime path

```
git clone https://github.com/gokulrajmisox/ews.git
cd ews
python -m venv .venv
source .venv/bin/activate
make install
python run.py --server
```

Open:

```
http://127.0.0.1:8000
http://127.0.0.1:8000/docs
http://127.0.0.1:8000/health
```

### 1.19.2 Full data-dependent path

Place the permitted raw dataset under `data/raw/`, then run:

```
python run.py --reproduce
```

This explicitly runs tests, training, calibration, chronological evaluation, ablation, and noise testing. It does not run implicitly during server startup.

## 1.20 Closing perspective

SilentWindow was built as a sequence of safeguards around a probabilistic model:

```
messy data
-> credibility assessment
-> patient-specific causal features
```

- > calibrated model risk
- > persistent evidence
- > bounded alert policy
- > auditable dashboard/API

The project's strongest contribution is not a claim of perfect prediction. It is the decision to make **data quality, temporal causality, persistence, alert burden, and limitations visible in the same system.**